

在流水线中添加普通用户态指令

在本章中，我们将介绍如何在已有的简单流水线 CPU 上添加更多的普通用户态指令。具体包括：

- **算术逻辑运算类指令**：slti、sltui、andi、ori、xori、sll、srl、sra、pcaddu12i，见 6.1 节。
- **乘除法运算类指令**：mul.w、mulh.w、mulh.wu、div.w、mod.w、div.wu、mod.wu，见 6.2 节。
- **转移指令**：blt、bge、bltu、bgeu，见 6.3 节。
- **访存指令**：ld.b、ld.h、ld.bu、ld.hu、st.b、st.h，见 6.4 节。

【本章学习目标】

- 加深对流水线结构与设计的理解。
- 掌握在 CPU 中增加普通用户态指令的方法。

【本章实践目标】

本章有两个实践任务（见 6.5 节）。读者可以在学习本章内容的基础上完成这些任务，两者之间的对应关系如下：

- 6.1 ~ 6.2 节的内容对应实践任务 10（6.5.1 节）。
- 6.3 ~ 6.4 节的内容对应实践任务 11（6.5.2 节）。

6.1 算术逻辑运算类指令的添加

添加指令的设计过程一般包括三个步骤：

- 1) 认真阅读指令系统规范, 明确待实现指令的功能定义。
- 2) 根据指令的功能定义考虑数据通路的设计调整, 能复用的尽量复用, 该新增的就新增。
- 3) 根据调整后的数据通路, 梳理所有指令 (包括原有的指令和新增的指令) 对应的控制信号。

对于本章即将要添加的 `slti`、`sltui` 等算术逻辑运算指令的功能定义, 请读者先自行阅读指令手册, 接下来的分析将假定各位读者已经熟知这部分内容了。

6.1.1 `slti` 和 `sltui` 指令的添加

我们将 `slti` 和 `slt` 指令进行比较, 会发现两者对两个源操作数的比较运算以及写回的目的寄存器是完全一致的, 差异仅在于两者的操作数来源。不过, 若将 `slti` 和 `addi.w` 指令进行比较, 则会发现两者的操作数来源是一样的。这就意味着 `slti` 指令的处理一方面可以复用 `slt` 指令在执行、访存和写回流水阶段的数据通路, 另一方面可以复用 `addi.w` 指令在译码流水阶段的数据通路。相应地, `slti` 指令在上述各阶段所需的控制信号也与所复用数据通路对应指令的控制信号一致。

采用相同的思路将 `sltui` 和 `sltu`、`addi.w` 指令进行比较分析, 可以得到设计思路是: `sltui` 指令的处理一方面可以复用 `sltu` 指令在执行、访存和写回流水阶段的数据通路, 另一方面可以复用 `addi.w` 指令在译码流水阶段的数据通路。相应地, `sltui` 指令在上述各阶段所需的控制信号也与所复用数据通路对应指令的控制信号一致。

6.1.2 `andi`、`ori` 和 `xori` 指令的添加

我们将 `andi`、`ori`、`xori` 指令和 `and`、`or`、`xor` 指令进行功能定义的比较, 发现它们进行的逻辑运算以及写回的目标寄存器号来源是相同的。因此, `andi`、`ori` 和 `xori` 指令可以复用 `and`、`or` 和 `xor` 指令在执行、访存和写回阶段的数据通路。相应地, 这些指令在这些流水阶段所需的控制信号与所复用数据通路对应指令的控制信号一致。

不过, `andi`、`ori` 和 `xori` 的第二个源操作数是对 12 位立即数 `ui12` 进行零扩展, 没有现成的数据通路可以复用, 需要对译码阶段的数据通路进行调整, 在生成第二个源操作数的多路选择器中添加一个“指令码立即数域 `ui12` 零扩展至 32 位”的输入。由于生成第二个源操作数的多路选择器的规格发生了变化, 因此不仅是 `andi`、`ori` 和 `xori`, 原先已实现的指令也要针对这个更新规格的多路选择器生成正确的控制信号。

6.1.3 sll.w、srl.w 和 sra.w 指令的添加

对于 sll.w、srl.w 和 sra.w 指令的添加，我们仍然采用上面与具有相似操作的指令进行比较分析的思路。分析后可知，sll.w、srl.w、sra.w 指令在执行、访存、写回阶段的数据通路可以分别复用 slli.w、srli.w 和 srai.w 指令的数据通路，而这三条指令在译码阶段的数据通路可以复用 add.w 指令的数据通路。余下的工作就是将这些指令在各阶段的控制信号置为与其所复用数据通路的指令一致。

6.1.4 pcaddu12i 指令的添加

通过分析 pcaddu12i 指令的定义，会发现该指令完成了一个加法运算且运算结果写入第 rd 项通用寄存器，这部分功能与 add.w 指令一致；该指令源操作数一 PC 的处理与 bl 指令计算 PC+4 时源操作数一的一致，源操作数二 {si20, 12'b0} 的处理与 lu12i.w 的一致。因此，pcaddu12i 指令一方面可以复用 add.w 指令在执行（不包含源操作数准备）、访存和写回流水阶段的数据通路，另一方面可以分别复用 bl 和 lu12i.w 指令在译码和执行（其中的源操作数准备）流水阶段的数据通路。相应地，pcaddu12i 指令所需的控制信号也与所复用数据通路对应指令的控制信号一致。

6.2 乘除法运算类指令的添加

对于整数补码乘法运算而言，两个 32 位的二进制数相乘之后会产生一个 64 位的二进制结果，由于通用寄存器堆每一项的宽度为 32 位，因此乘法的结果无法放入寄存器堆的一项中。因此，LoongArch32 精简版指令集中为完成整数乘法运算定义了 mul.w、mulh.w 和 mulh.wu 三条指令。其中 mulh.w 和 mul.w 指令均进行两个 32 位有符号整数的乘法运算，分别将乘积的高 32 位和低 32 位写入目的通用寄存器 rd 中；mulh.wu 和 mul.w 指令均进行两个 32 位无符号整数的乘法运算，分别将乘积的高 32 位和低 32 位写入目的通用寄存器 rd 中。之所以有符号数乘法和无符号数乘法乘积的低 32 位都是用 mul.w 指令，是因为当输入数据的二进制值一致时，有符号数乘和无符号乘的结果的低半部分的二进制值是相同的。

对于整数补码除法运算而言，两个 32 位的二进制数相除之后会产生一个 32 位的商和一个 32 位的余数。LoongArch32 精简版指令集中定义了 div.w、mod.w 指令，分别计算两个有符号数相除的商和余数，定义了 div.wu、mod.wu 指令，分别计算两个无符号

数相除的商和余数。

上述乘除法指令，两个源操作数均分别来自 `rj` 和 `rk` 寄存器，计算的结果都是写入到 `rd` 寄存器，与 `add.w` 指令比较可知其相同，因此在译码、写回级流水阶段可以复用 `add.w` 的数据通路并生成相同的控制信号。数据通路中主要的设计调整是增加乘、除运算部件。我们接下来将给出两种不同实现难度的乘、除法运算部件的设计方法。第一种方法是采用调用 Xilinx IP 的方法（见 6.2.1 节），第二种方法是在逻辑门电路层次自行实现乘法运算单元（见 6.2.2 节）和除法运算单元（见 6.2.3 节）。显然，后者需要花费的设计和调试时间远高于前者，不过能学到的东西也更多，读者可根据个人喜好及学习进度决定是否采用第二种方法。

6.2.1 调用 Xilinx IP 实现乘除法运算部件

6.2.1.1 调用 Xilinx IP 实现乘法运算部件

调用 Xilinx IP 实现乘法运算部件的最简单方法是采用如下形式的代码：

```
wire [31:0] src1, src2;
wire [63:0] unsigned_prod, signed_prod;

assign unsigned_prod = src1 * src2;
assign signed_prod   = $signed(src1) * $signed(src2);
```

Vivado 中的综合工具遇到上面代码中的 “*” 运算符时，会像遇到 “+” “>>” 运算符一样，将根据设计给出的时序等约束情况，从自身的 IP 库中找到一个适合的乘法器电路，将其嵌入到你的设计中。根据我们的实验，对于 Artix-7 系列的 FPGA，目前 Vivado 实现 “*” 运算符时会默认采用 DSP48 器件（内含固化的 16 位乘法器电路），所以最终实现的电路的时序通常不错，也几乎不消耗 LUT 资源。采用这种方式推导出的乘法器电路有两个 32 位数输入、一个 64 位输出，具有单周期延迟，即输入数据之后当拍就能输出结果。

不过，从给出的参考代码中大家也会发现，其中推导出的 Xilinx 的乘法器 IP 要么只能做有符号乘法，要么只能做无符号乘法，所以为了实现 `mulh.w` 和 `mulh.wu` 指令，我们要实例化两个乘法器 IP。这个方法看起来不那么“高大上”。其实 32 位有 / 无符号乘法可以统一转化为 33 位有符号乘法，方法是将 32 位有 / 无符号数转化为 33 位有符号数：对 32 位有符号数，最高位扩展为符号位；对 32 位无符号数，最高位补 0。这样处理后得到的 66 位乘积结果的最高 2 位可以不管，仍然根据指令的定义选取结果的

[63:32] 或 [31:0] 位。

6.2.1.2 调用 Xilinx IP 实现除法运算部件

我们这里不使用“/”和“%”这类运算符来让综合工具推导实现除法运算部件，主要原因是 Vivado 综合工具将用 LUT 实现一个单周期的除法运算部件，其时序会非常差。接下来介绍的是基于交互界面定制除法器 IP 的方法。

在工程导航栏中点击“IP Catalog”，然后在右侧出现的窗口中搜索 div，如图 6.1 所示。

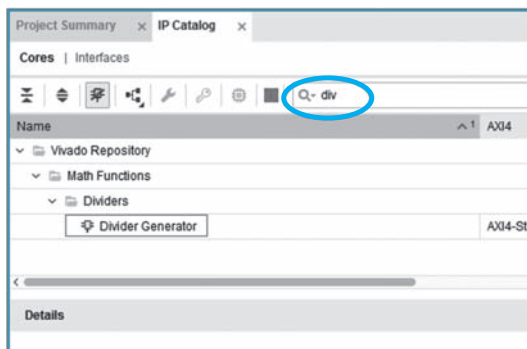


图 6.1 在“IP Catalog”界面中选择“Divider Generator”

在搜索结果中双击“Divider Generator”项，打开除法器 IP 设置界面，依次设置参数，如图 6.2 所示。

对于除法器 IP 设置选项中需要大家关注的地方，我们在图中做了标记和说明。对于除法器的名字，读者可以根据自己的喜好进行调整，并不一定要采用图中的示例名。在“Channel Settings”选项卡中，我们首先要对除法器 IP 的算法进行选择，这里建议采用 Radix2 算法。采用 Radix2 算法实现的除法器的优点是资源消耗少，缺点是完成运算需要的迭代周期数多。通常，应用程序中出现定点除法运算的比例很低，而且编译器一般会尽可能用加、减、移位等指令来实现除法运算，所以除法指令实现的周期数略多一些，但对应用的平均性能不会造成太大的影响。在“Channel Settings”选项卡中还可以选择除法运算是有符号除法还是无符号除法。与前面介绍的乘法器 IP 类似，这里也需要生成有符号和无符号两个除法器 IP，才能分别用于实现 div.w/mod.w 和 div.wu/mod.wu 指令。接下来的 5、6 两处分别定义被除数和除数的位宽为 32 位。在 7 处，一定要将 Remainder Type 选择为 Remainder，以确保余数类型是整型。我们不用勾选 8 处的除 0 检测，因为 LoongArch 指令规范中规定除法指令自身是不需要对除数为 0 进行特

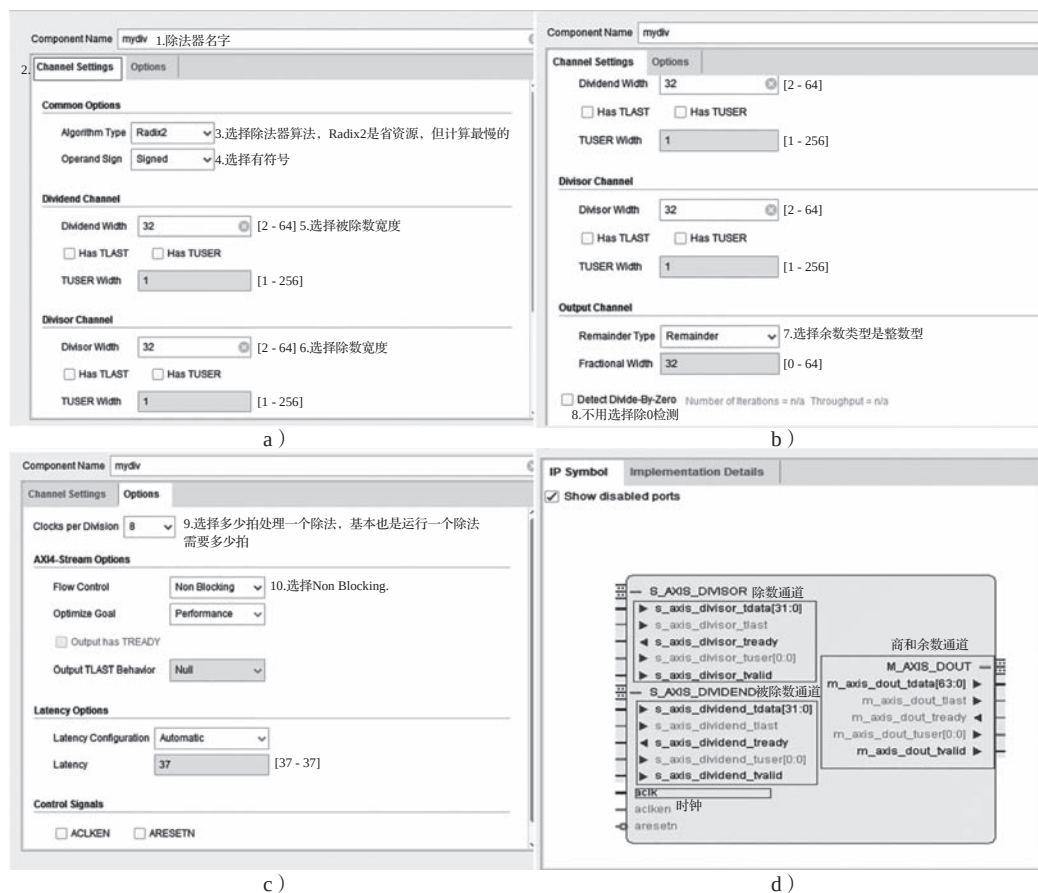


图 6.2 设置除法器 IP 核的参数

殊处理的，这项工作交给软件去做。在“Options”选项卡中，9处选择多少周期处理一个除法，这个拍数是连续处理多个除法运算之间的间隔周期数，不是完成一个除法运算的周期数。完成一个除法运算完成的周期数由图 6.2c 中的“Latency Options”参数决定。10处 AXI 接口上的流控可以按照我们的建议选择 Non Blocking，这样选择意味着除法器产生计算结果后，外部一定能够取走这个结果，不会阻塞结果的输出。在我们的单发射静态流水线中，这个条件是始终成立的。（请读者自己想一想其中的原因。）

尽管选择 Non Blocking 的流控策略，但由于目前 Vivado 中提供的除法器 IP 一定是 AXI 接口的，因此接下来我们还是要对模块顶层接口的相关信号进行一些介绍。观察图 6.2d，总体上我们会看到被除数、除数通道以及商和余数通道。其中，被除数通道中的 32 位输入信号 s_axis_dividend_tdata 对应计算时输入的被除数数据；除数通道中的 32 位输入信号 s_axis_divisor_tdata 对应计算时输入的除数数据；计算得到的商和余数

结果统一从商和余数通道中的 64 位信号 `m_axis_dout_tdata` 中输出，其中第 [63:32] 位存放的是商，第 [31:0] 位存放的是余数。

对于两个输入通道和一个输出通道，除了有上述数据信号外，每个通道都有一对 `tvalid`、`tready` 信号。这是一对“握手”控制信号，其工作原理类似于我们在 CPU 流水线之间使用的 `valid`、`allowin` 信号。`tvalid` 是请求信号，`tready` 是应答信号。在时钟上升沿到来时，如果采样得到 `tvalid` 和 `tready` 都等于 1，则请求发起方和接收方之间完成一次成功的握手。如果假想接收方有一组触发器缓存，那么所谓的成功握手是指发送方的数据写入接收方的缓存中，也就是在握手成功的这个上升沿之后，触发器缓存会变为发送方的数据。

假设在执行流水阶段调用所生成的除法器 IP。在除法指令处于执行流水级且没有对除法器成功输入数据的时候，同时将 `s_axis_dividend_tvalid` 和 `s_axis_divisor_tvalid` 置为 1。当发现 `s_axis_dividend_tready` 和 `s_axis_divisor_tready` 反馈为 1 后（此时在一个时钟上升沿同时看到 `tvalid` 和 `tready` 为 1，表示握手成功），需要将 `s_axis_dividend_tvalid` 和 `s_axis_divisor_tvalid` 清 0，也就是确保握手成功的那个时钟上升沿之后的 `s_axis_dividend_tvalid` 一定同时置为 1，那么这里生成的除法器 IP 反馈的 `s_axis_dividend_tready` 和 `s_axis_divisor_tready` 一定也同时置为 1。这里再次强调，被除数和除数输入的 `tready` 置起后（也就是握手成功后），`tvalid` 一定要撤销，否则除法器 IP 会认为又有一个新的除法运算。

完成输入数据的握手之后，除法指令就需要在执行流水级等待除法器 IP 最终输出结果。当 `m_axis_dout_tvalid` 置为 1 时，表示除法计算完成。此时除法指令就可以从 `m_axis_dout_tdata` 上取出计算结果，进入流水线的后续阶段。由于前面我们将流控策略设置为 Non Blocking，因此输出通道上不需要外部反馈 `tready` 信号，换言之，不管产生多少结果、什么时候产生，外部都要能够及时处理。

采用本小节介绍的实现方案，尽管除法是在 CPU 的执行流水阶段开始执行并产生运算结果，但由于其运算会花费多个时钟周期，所以流水线的控制上要做一些针对性的调整。具体来说，就是当执行流水阶段上是一条除法指令时，仅当除法运算部件返回结果完成的信号（Xilinx 除法器 IP 的 `m_axis_dout_tvalid` 信号为 1）之后这一级流水的 `ready_go` 才能置为有效。

如果读者不想在 CPU 设计中将更多的时间花费在乘除法运算电路的设计实现上，那么按照这一节介绍的方法去实现就可以了，然后直接跳过接下来的 6.2.2 节和 6.2.3 节。

6.2.2 电路级实现乘法器

移位乘法器和我们平时使用的列竖式计算乘法的方式相同,即将 32 位乘法转化为 32 个 64 位数的加法,再计算出结果。值得注意的是,两个数的补码的积并不等于积的补码。如果 $[Y]_{\text{补}} = y_{31}y_{30}\cdots y_1y_0$, 则

$$[X \times Y]_{\text{补}} = [X]_{\text{补}} \times (-y_{31} \times 2^{31} + y_{30} \times 2^{30} + \cdots + y_1 \times 2^1 + y_0 \times 2^0)$$

也就是说 $[X \times Y]_{\text{补}}$ 并不等于 $[X]_{\text{补}} \times [Y]_{\text{补}}$, 而是等于把 $[Y]_{\text{补}}$ 的最高位取反, 其他位不变的数和 $[X]_{\text{补}}$ 相乘的结果。更详细的推导过程请参看《计算机体系结构基础》(第 3 版) 的 8.3 节。以一个 4 位的乘法器为例: $0101(5) \times 1001(-7) = 1011101(-35)$ 。简单补码的乘法运算如图 6.3 所示(每个部分积均要做符号扩展):

				0	1	0	1	
				×	1	0	0	1
+	0	0	0	0	1	0	1	
+	0	0	0	0	0	0		
+	0	0	0	0	0			
−	0	1	0	1				
	1	0	1	1	1	0	1	

图 6.3 简单补码乘法运算过程举例

在补码乘法运算过程中, 只要对 Y 的最高位乘积项做减法, 对其他位乘积项做加法即可。但在做加法和减法操作时, 一定要扩充符号位, 使乘积项对齐。

可以使用 1 位 Booth 算法进行补码乘法, 将各部分积统一成相同的形式。

6.2.2.1 2 位 Booth 编码

对 32 位乘法而言, 无论是上述简单的补码乘法器还是 1 位 Booth 算法实现的乘法器, 都需要将 32 个部分积相加才能得到结果, 延迟和硬件的开销都很大。引入 2 位 Booth 编码可以显著减少加法的次数。

对

$$(-y_{31} \times 2^{31} + y_{30} \times 2^{30} + \cdots + y_1 \times 2^1 + y_0 \times 2^0)$$

做如下变换:

$$\begin{aligned} & (-y_{31} \times 2^{31} + y_{30} \times 2^{30} + \cdots + y_1 \times 2^1 + y_0 \times 2^0) \\ &= (y_{29} + y_{30} - 2 \times y_{31}) \times 2^{30} + (y_{27} + y_{28} - 2 \times y_{29}) \times 2^{28} + \cdots + \\ & \quad (y_1 + y_2 - 2 \times y_3) \times 2^2 + (y_0 - 2 \times y_1) \times 2^0 \\ &= (y_{29} + y_{30} - 2 \times y_{31}) \times 2^{30} + (y_{27} + y_{28} - 2 \times y_{29}) \times 2^{28} + \cdots + \\ & \quad (y_1 + y_2 - 2 \times y_3) \times 2^2 + (y_{-1} + y_0 - 2 \times y_1) \times 2^0 \end{aligned}$$

其中 y_{-1} 取值为 0。

根据上式, 可以先将 Y 的 -1 位补 0, 然后每次扫描 Y 的 3 位来确定乘积项, 这样乘积项减少了一半, 32 位的乘法只需 15 次加法。2 位 Booth 编码的规则如表 6.1 所示。

表 6.1 2 位 Booth 乘法运算编码规则

y_{i+1}	y_i	y_{i-1}	操作
0	0	0	不需要加 (+0)
0	0	1	补码加 X ($+[X]_{\text{补}}$)
0	1	0	补码加 X ($+[X]_{\text{补}}$)
0	1	1	补码加 $2X$ ($+[X]_{\text{补}}$ 左移)
1	0	0	补码减 $2X$ ($-[X]_{\text{补}}$ 左移)
1	0	1	补码减 X ($-[X]_{\text{补}}$)
1	1	0	补码减 X ($-[X]_{\text{补}}$)
1	1	1	不需要加 (+0)

比如, $0101(5) \times 1001(-7)$, 其中 $[X]_{\text{补}} = +0101$, $2[X]_{\text{补}} = +01010$, $-[X]_{\text{补}} = -0101 = +1011$, $-2[X]_{\text{补}} = -01010 = +10110$, 计算过程如图 6.4 所示。

				0	1	0	1	
				×	1	0	0	1
								0
+	0	0	0	0	1	0	1	
+	1	0	1	1	0			
	1	0	1	1	1	0	1	

(补 y_{-1})
(010, $+[X]_{\text{补}}$)
(100, $-2[X]_{\text{补}}$)
(结果为-35补码)

图 6.4 2 位 Booth 编码补码乘法运算过程举例

可以看到, 对 4 位乘法使用 2 位 Booth 编码之后, 部分积由原来的 4 个减少为 2 个, 需要加法的次数由原来的 3 次减少为 1 次, 极大地提高了效率。

6.2.2.2 保留进位加法器

对于 32 位乘法, 即使使用了 2 位 Booth 编码, 16 个部分积仍然需要进行 15 次接近 64 位的加法操作。一般做一次 65 位加法已经会有很大延迟了。如果做 15 次加法, 使用累加的形式, 则效率会更低。对于多个数的相加, 可以考虑使用不需要等待进位信号的保留进位加法器。

保留进位加法器就是使用全加器将三个加数的加法转化成两个加数的加法的装置, 转化后的两个加数中, 有一个是所有的本地和组成的, 另一个是所有的向高位的进位信号组成的。图 6.5 给出了保留进位加法器的运算过程。

				1	0	1	1	0	1	0	1	
				1	1	1	0	1	1	1	0	
+				1	0	1	1	1	1	0	0	
				1	1	1	1	0	0	1	1	
												高位补符号位
+				1	0	1	1	1	1	0	0	0
												低位补0

图 6.5 补码乘法运算保留进位加法举例

$$10110101(-75) + 11101110(-18) + 10111100(-68) = 111100111(-25) + 101111000(-136)$$

保留进位加法器的每一位的计算都是独立的, 不会依赖其他位的信息, 所以不会产生进位延迟。例子中的最低位的 3 个加数为 1、0、0, 结果为 1, 进位为 0; 最高位的 3 个加数为 1、1、1, 结果为 1, 进位为 1, 其他位也是一样, 最后还需将进位结果左移一位。

6.2.2.3 华莱士树

对于要将很多个加数相加得到一个结果的运算, 可以先使用一层保留进位加法器(也就是一组全加器)将其转化为约 2/3 个加数相加, 再使用一层保留进位加法器转换为约 4/9 个加数相加, 直到最后转化为 2 个加数相加。这样的构造叫作**华莱士树**。

使用 2 位 Booth 编码后, 32 位乘法被转化为 16 个 64 位的部分积相加。可以使用六层华莱士树将加数减少为 2 个, 图 6.6 是 16 个部分积中, 针对某 1 位构建的华莱士树。

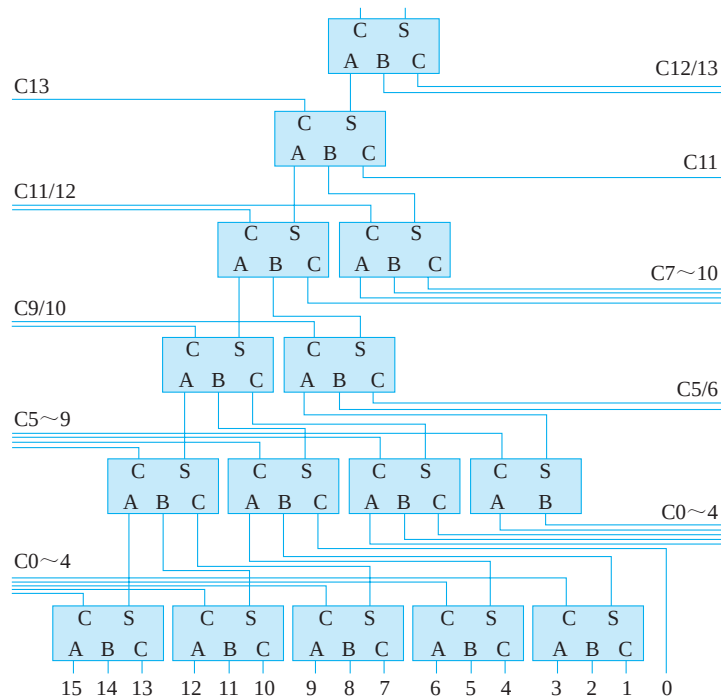


图 6.6 16 个数相加的 1 位华莱士树

6.2.2.4 使用 2 位 Booth 编码 + 华莱士树的 32 位补码乘法器

在 Booth 编码和华莱士树的基础上不难设计出 32 位定点补码乘法器, 其结构如图 6.7 所示。

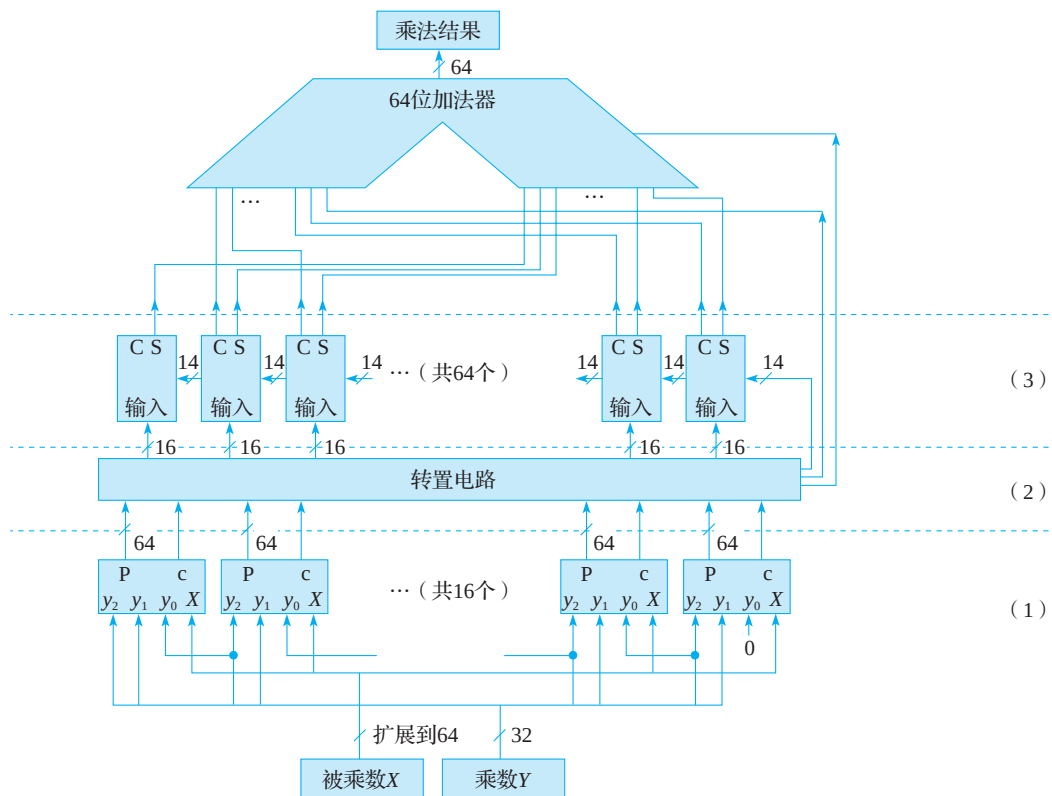


图 6.7 32 位定点补码乘法器结构

图 6.7 中的第 1 部分采用 2 位 Booth 算法得到 16 个部分积，其中 P 为 64 位，是部分积的主体； c 为 1 位，表示对被乘数取反。

第 2 部分类似矩阵转置，将 16 个 64 位部分积转置为 64 个 16 位等宽的数，用作华莱士树每位处的输入，这一部分在电路中的体现就是连线，没有任何多余的电路单元。

第 3 部分为华莱士树，是 64 个 1 位的华莱士树的集合。需要注意的是，每 1 位的华莱士树有来自低位的进位信号，而最高位处的华莱士树向高位的进位则被忽略了。这是因为 32 位有符号数乘以 32 位有符号数能得到的最大的数是 $-2^{31} \times -2^{31} = 2^{62}$ ，用 64 位有符号数完全可以容纳这个结果，就算将最高位处的华莱士树向高位的进位代入运算，得到的也是符号位的扩展，所以可以直接省略。

6.2.2.5 对乘法器的进一步优化

1. 用定点补码乘法器实现无符号乘法

上述几节介绍的都是补码乘法器，显然这是针对有符号乘法的。而在指令集里，除

了有符号乘法之外, 还有无符号乘法, 所以我们实现的乘法器还需要支持无符号乘法。

所谓有符号乘法和无符号乘法, 是指进行相乘的源操作数是被当作有符号数还是无符号数, 比如 32 位寄存器里存放的数为 $0x80000000$, 如果该数被看作有符号数, 则是 -2^{31} (计算机里存储的有符号数均为补码形式); 如果它被看作无符号数, 则是 2^{31} 。

虽然无符号数与有符号数看起来差别很大, 但是只要在没有符号数的最高位前再补一个 0, 就可以把它看作符号位为 0 的有符号数。同时, 对于有符号数, 在最高位前再扩展一位符号位, 则表示的数值不变。通过这种最高位前再补一位的方法就可以将有符号数和无符号数统一, 相应地, 32 位数就扩展成了 33 位数。

因此, 可以实现一个 33 位的有符号乘法器, 使得它既可以做有符号乘法运算, 又可以做无符号乘法运算。每次运算前需要将源操作数扩展为 33 位, 对于有符号数, 在最高位前补符号位; 对于无符号数, 在最高位前补 0。比如, 对于 32 位数 $0x8000_0000$, 将它当作无符号数时需扩展为 33 位 $0x0_8000_0000$, 将它当作有符号数时需扩展为 $0x1_8000_0000$ 。需要注意的是, 33 位数经过 2 位 Booth 编码后会产生 17 个部分积, 相应的华莱士树的结构也要做调整, 如图 6.8 所示。

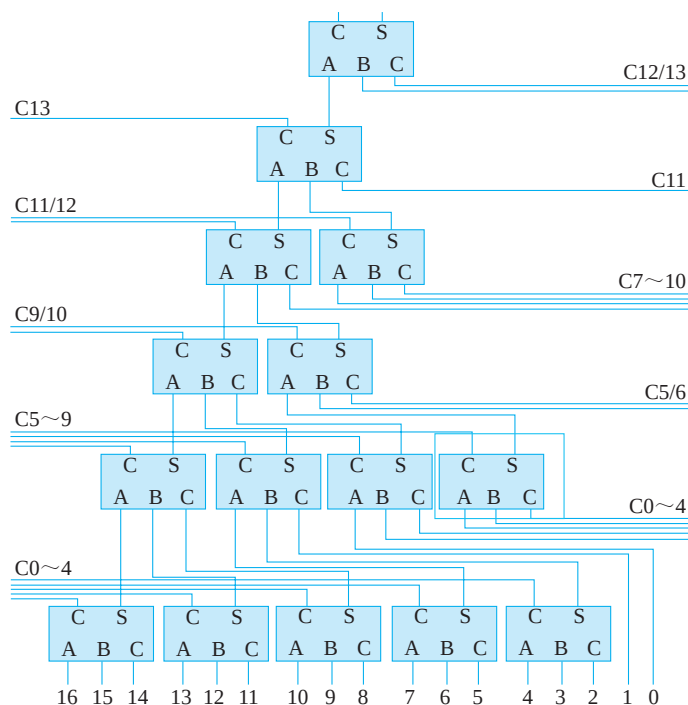


图 6.8 17 个部分积相加的 1 位华莱士树

2. 流水化改进

按照上面的描述设计出的单周期 32 位补码乘法器，经过 Vivado 工具综合得到的最长延迟时间约为 36ns，延时过长，所以可以考虑将该乘法器进行流水化改进，即将其切分为前后相连的多个流水级。流水化改进后，在能基本保证每个周期执行一条乘法的前提下^①，提高整个乘法器的运行频率。

切分流水线要尽量做到均衡，即被流水线分隔的几个部分的延迟时间差距不能太大。比如，将乘法器切分成两级流水结构，应该尽量使切分流水之后的最长延迟时间接近单周期时的一半。可以先根据乘法器的结构来确定流水线的位置，切分完成后再使用综合工具进行评估。两级流水线的乘法器可以在 CPU 中占据执行级和访存级的位置，并不会拖延整个 CPU 的运行节奏。

虽然要求乘法器切分为两级，但建议还是将乘法器写为一个单独的模块，这样，替换整个乘法器、对乘法器进行模块级验证也很方便。推荐乘法器模块实现的主要接口如表 6.2 所示。

表 6.2 推荐实现的乘法器模块的主要接口

信号	位宽	方向	功能
mul_clk	1	input	乘法器模块时钟信号
resetrn	1	input	复位信号，低电平有效
mul_signed	1	input	控制有符号乘法和无符号乘法
x	32	input	被乘数
y	32	input	乘数
result	64	output	乘法结果

切分的流水级也包含在乘法器模块中，该模块的输入信号均来自执行级，输出信号在访存级被获取，进行前递或传入到写回级，故乘法器模块内部只有一组流水级间的寄存器。

6.2.2.6 乘法器模块级验证

在实现乘法模块后，最好对其进行模块级随机验证，确保功能正确。

对于切分成两级流水结构的乘法器模块，可以使用 mycpu_env/module_verify/mul_verify/ 目录下的模块级验证环境来验证，其 testbench 顶层 mul_tb 如下：

```
`timescale 1ns / 1ps
```

① 基本保证是因为如果相邻乘法操作之间存在写后读相关，那么无法每个时钟周期都开始一个新的执行。

```

module mul_tb;

    // 输入
    reg mul_clk;
    reg resetn;
    reg mul_signed;
    reg [31:0] x;
    reg [31:0] y;

    // 输出
    wire signed [63:0] result;

    // 实例化 UUT(Unit Under Test )
    mul uut (
        .mul_clk(mul_clk),
        .resetn(resetn),
        .mul_signed(mul_signed),
        .x(x),
        .y(y),
        .result(result)
    );

    initial begin
        // 初始化输入
        mul_clk = 0;
        resetn = 0;
        mul_signed = 0;
        x = 0;
        y = 0;
        #100;
        resetn = 1;
    end
    always #5 mul_clk =~mul_clk;

    // 产生随机乘数和有符号控制信号
    always @(posedge mul_clk)
    begin
        // $random 为系统任务，产生一个随机的 32 位有符号数
        x          <= $random;
        y          <= $random;
        // 加了拼接符，{$random} 产生一个非负数，除 2 取余得到 0 或 1
        mul_signed <= {$random}%2;
    end

    // 寄存乘数和有符号乘控制信号，因为是两级流水，故存一拍
    reg [31:0] x_r;
    reg [31:0] y_r;
    reg          mul_signed_r;

```



```

always @(posedge mul_clk)
begin
    if (!resetn)
    begin
        x_r          <= 32'd0;
        y_r          <= 32'd0;
        mul_signed_r <= 1'b0;
    end
    else
    begin
        x_r          <= x;
        y_r          <= y;
        mul_signed_r <= mul_signed;
    end
end
end

// 参考结果
wire signed [63:0] result_ref;
wire signed [32:0] x_e;
wire signed [32:0] y_e;
assign x_e          = {mul_signed_r & x_r[31],x_r};
assign y_e          = {mul_signed_r & y_r[31],y_r};
assign result_ref   = x_e * y_e;
assign ok           = (result_ref == result);

// 打印运算结果
initial begin
    $monitor("x = %d, y = %d, signed = %d, result = %d,OK=%b",
            x_e,y_e,mul_signed_r,result,ok);
end

// 判断结果是否正确
always @(posedge mul_clk)
begin
    if (!ok)
    begin
        $display("Error: x = %d, y = %d,result = %d, result_ref = %d, OK=%b",
            x_e,y_e,result,result_ref,ok);
        $finish;
    end
end
end

endmodule

```

注意，声明和计算有关的变量时必须写上 signed 以表示有符号数。testbench 中如果显示 OK=1，则说明结果正确，否则说明运算有错，同时仿真会停止。

6.2.3 电路级实现除法器

根据是否将源操作数转换为绝对值，除法器可分为绝对值除法器 and 补码除法器。通常，绝对值除法器的结果是商和余数的绝对值，因而最后需要计算商和余数的补码；而补码除法器虽然得到的结果是补码，但计算过程中可能存在多减除数的情况，所以需要调整对余数进行调整。

6.2.3.1 迭代除法器

除法器通常是需要迭代进行的，简单的迭代除法是试商法，32 位除法的商最多也是 32 位，故可以依次从商的第 31 位到第 0 位试 0 或 1，这和笔算除法的方法类似。

根据迭代过程中，在不够减时（商为 0）是否恢复余数可分为恢复余数法（循环减法）和不恢复余数法（加减交替）。加减交替是指将不够减的情况视为减多了，但此时并不恢复余数，而是在下一次迭代改为加法，以补回多减的。该方法的具体操作是，假设第二轮迭代采用加法余数为 r_2+x ，其中 r_2 为第一轮迭代中通过减法得到的余数 r_1-2x ，所以第二轮迭代的余数就相当于 r_1-x ，与采用恢复余数法得到的结果是一样的。对于为什么前一次减法减去的是 $2x$ （是后一次的两倍），可以假设被除数是 $abcd$ （四位），除数是 yz （两位），那么从高位向低位迭代，开始是 $ab-yz$ ，相当于 $ab00-yz00$ ；第二次是 $bc-yz$ ，相当于 $bc0-yz0$ 。也就是说，第一次是 $2x$ ，第二次是 x 。这也是迭代除法实现时使用移位器的道理。

其实，对于所有迭代除法器，绝对值、补码除法器，或者循环减法、加减交替等都是基于一个原理：判断剩余被除数和除数的大小，确定商，更新剩余被除数。只是在具体实施中，根据不同的处理方法得到上述除法器分类。

比如，对于补码除法器，在有符号除法的情况下，假设被除数为 `0xffff_ffff`，除数为 `0x1111_1111`，也就是被除数是负数，除数是正数。显然，比较被除数和除数的大小是需要使用加法操作的。这时如果采用恢复余数法，那就是循环加法补码除法器；如果采用不恢复余数法，在迭代过程中，如果将余数加成了正数，那就说明加多了（绝对值减多了），就需要改为减法，这就是加减交替补码除法器。

以 1 位恢复余数绝对值迭代除法器为例，运算过程分为三步。

步骤 1：根据被除数和除数确定商和余数的符号，并计算被除数和除数的绝对值。

根据指令手册中指令的定义，余数的符号要和被除数的符号保持一致，这样商和余数的符号就可以由表 6.3 确定。

表 6.3 除法结果的符号位规则表

被除数	除数	商	余数
正	正	正	正
正	负	负	正
负	正	负	负
负	负	正	负

对于无符号数，计算机里保存的数就是其原码，故绝对值就是该数。对于有符号数，如果该数为正数，则计算机里保存的数为其补码，也是其原码，故绝对值就是该数；如果该数是负数，则计算机里保存的为其补码，直接对该补码（含符号位）取反加 1 即得到其绝对值。

步骤 2：迭代运算得到商和余数的绝对值。

迭代运算的过程如下：

- 1) 在 32 位的被除数前面补 32 个 0，记为 A[63:0]，并记除数为 B[31:0]，得到的商记为 S[31:0]，余数记为 R[31:0]。
- 2) 第一次迭代，取 A 的高 33 位，即 A[63:31]，与 B 的高位补 0 的结果 {1'b0, B[31:0]} 做减法：如果结果为负数，则商的相应位（S[31]）为 0，被除数保持不变；如果结果为正数，则商的相应位记为 1，将被除数的相应位（A[63:31]）更新为减法的结果。
- 3) 进行第二次迭代，此时就需要取 A[62:30] 与 {1'b0, B[31:0]} 做减法，依据结果更新 S[30]，并更新 A[62:30]。
- 4) 依此类推，直到算完第 0 位。

步骤 3：调整最终的商和余数。

步骤 2 中得到的商和余数为对应的绝对值，根据步骤 1 中确定的商和余数符号将商和余数转化为有符号数，提交结果。

下面以 8 位补码除法 $10010101 (-107) \div 00011101 (29)$ 为例介绍迭代除法。

首先，因为被除数为负、除数为正，所以确定商为负、余数为负，并且计算得到被除数的绝对值为 01101011，除数的绝对值为 00011101。之后进入迭代减法的操作。

第 1 次迭代得到商的最高位，如图 6.9 所示。

0															
0	0	0	0	0	0	0	0	0	1	1	0	1	0	1	1
-	0	0	0	0	1	1	1	0	1						
1	1	1	1	0	0	0	1	1							
0	0	0	0	0	0	0	0	0	1	1	0	1	0	1	1

图 6.9 除法运算举例第 1 次迭代

图 6.9 中第 1 行为商的绝对值, 第 2 行为第一拍执行之前被除数的绝对值, 第 3 行为除数的绝对值, 第 4 行为减法的结果, 最后一行是第一拍执行之后被除数的绝对值。

在前 6 次迭代中, 被除数的绝对值都没有发生变化, 所以略去这些迭代过程。直接看第 7 次迭代, 结果如图 6.10 所示。

										0	0	0	0	0	0	1	
	0	0	0	0	0	0	0	0	0	1	1	0	1	0	1	1	
-										0	0	0	0	1	1	0	1
										0	0	0	0	1	1	0	0
	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1	

(减除数)
(够减, 商上1)
(调整剩余被除数)

图 6.10 除法运算举例第 7 次迭代

第 8 次迭代, 得到最终商和余数的绝对值, 如图 6.11 所示。

											0	0	0	0	0	0	1	1		
	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	1				
-											0	0	0	0	1	1	1	0	1	(減除数)
											0	0	0	0	1	0	1	0	0	(够减, 商上1)
	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0			(调整剩余被除数)

(减除数)
(够减, 商上1)
(调整剩余被除数)

图 6.11 除法运算举例第 8 次迭代

于是得到商的绝对值为 00000011, 余数的绝对值为 00010100 (取低 8 位)。

最后, 将商和余数转为补码。因为前面已经确定了商为负数, 余数也为负数, 所以得到商 11111101 (-3) 和余数 11101100 (-20)。

这里要提醒读者的是: 对一个负的有符号数取绝对值就是将这个数“取反加 1”得到的结果, 对一个正的有符号数取绝对值的结果就是其本身; 对一个绝对值求其负的有符号数的结果也是“取反加 1”, 对一个绝对值求其正的有符号数的结果就是其本身。

从上述计算过程可以看到, 除法的商的寄存器是从高位到低位依次得到, 这和一个 32 位左移寄存器等效, 每次迭代得到的商放在第 0 位上; 被除数从高到低依次取 33 位与除数相减, 并更新这 33 位, 等效于一个 64 位左移寄存器, 每次取高 33 位与除数进行相减判断并更新这 33 位。

可以根据以上描述画出迭代除法器的结构示意图。

6.2.3.2 用迭代除法器进行无符号除法

如果仅仅让迭代除法器执行第 2 步的操作, 它就是一个无符号除法器。所以, 只需在迭代除法器中加入一个控制信号, 使除法在第 1 步和第 3 步中不做数值调整, 那么得

到的结果就是无符号除法的结果。这样除法器就可以进行无符号运算。

6.2.3.3 迭代除法器中的控制信号

迭代除法器运算的第 1 步为获取操作数和控制信号，第 3 步为输出结果到下一流水级，这两步各需要一拍。第 2 步为迭代运算，需要 32 拍。这样，一次除法需要 34 拍。可以在除法器中内置一个计数器，该计数器在除法开始时从 0 开始计数，当计数到 33 时，将除法完成信号置为 1 并停止计数，直到下一个除法开始。

同乘法器一样，建议将除法器也单独封装为一个模块，推荐实现的主要接口如表 6.4 所示。

表 6.4 推荐实现的除法器模块的主要接口

信号	位宽	方向	功能
div_clk	1	input	除法器模块时钟信号
resetsn	1	input	复位信号，低电平有效
div	1	input	除法运算命令，在除法完成后，如果外界没有新的除法进入，必须将该信号置为 0
div_signed	1	input	控制有符号除法和无符号除法的信号
x	32	input	被除数
y	32	input	除数
s	32	output	除法结果，商
r	32	output	除法结果，余数
complete	1	output	除法完成信号，除法内部 count 计算达到 33

6.2.3.4 除法器进一步优化

在以上例子中，除法都是一位试商，其实可以考虑两位试商，但代价就是时序变差、资源消耗更多。

可以观察以下除法：

- 1) $0x7777_7777/0x7777_7776$ ，会发现前面 31 位商都是 0。
- 2) $0x2000_0001/0x2$ ，会发现后面 30 位商都是 0。
- 3) $0x2000_0003/0x2$ ，会发现中间 29 位商是 0。

我们看到，迭代除法中经常会出现一串 0，在软件程序中，商的首部出现一串 0 的情况十分普遍。所以，可以考虑提前开始或提前结束的除法，中间有一串 0 的情况有可能加速迭代，但需要一定的设计技巧。

注意，所有的除法实现方法考虑的主要因素都是便于在硬件上实现，我们认为很简

单的很多操作，其硬件实现却比较复杂。比如，在一个 32 位数中确定首部连续 0 的个数，也就是查找一个 32 位数的前导 0，这个问题看起来很简单，但其硬件实现要花费很多资源。

6.2.3.5 除法器模块级验证

在除法模块实现后，最好对其进行模块级随机验证，确保功能正确。可以使用 my-cpu_env/module_verify/div_verify/ 目录下的模块级验证环境来验证，其 testbench 顶层 div_tb 如下：

```
`timescale 1ns / 1ps

module div_tb;

    // 输入
    reg div_clk;
    reg resetn;
    wire div;
    reg div_signed;
    reg [31:0] x;
    reg [31:0] y;

    // 输出
    wire [31:0] s;
    wire [31:0] r;
    wire complete;

    // 实例化 UUT (Unit Under Test)
    div uut (
        .div_clk(div_clk),
        .resetn(resetn),
        .div(div),
        .div_signed(div_signed),
        .x(x),
        .y(y),
        .s(s),
        .r(r),
        .complete(complete)
    );
    initial begin
        // 初始化输入
        resetn = 0;
        #100;
        resetn = 1;
    end
    initial
    begin
        div_clk = 1'b0;
```

```

    forever
    begin
        #5 div_clk = 1'b1;
        #5 div_clk = 1'b0;
    end
end

// 产生除法命令, 正在进行除法
reg div_is_run;
integer wait_clk;
initial
begin
    div_is_run <= 1'b0;
    forever
    begin
        @(posedge div_clk);
        if (!resetn || complete)
        begin
            div_is_run <= 1'b0;
            wait_clk <= {$random}%4;
        end
        else
        begin
            repeat (wait_clk)@(posedge div_clk);
            div_is_run <= 1'b1;
            wait_clk <= 0;
        end
    end
end

// 随机生成有符号除法控制信号、除数和被除数
assign div = div_is_run;
always @(posedge div_clk)
begin
    if (!resetn || complete)
    begin
        div_signed <= 1'b0;
        x <= 32'd0;
        y <= 32'd1;
    end
    else if (!div_is_run)
    begin
        div_signed <= {$random}%2;
        x <= $random;
        y <= $random; // 被除数随机产生 0 的概率很小, 基本可忽略
    end
end

//-----{ 计算参考结果 }begin
// 第一步, 求 x 和 y 的绝对值, 并判断商和余数的符号
wire x_signed = x[31] & div_signed; // x 的符号位, 做无符号除法时认为是 0

```



```

wire y_signed = y[31] & div_signed; //y 的符号位, 做无符号除法时认为是 0
wire [31:0] x_abs;
wire [31:0] y_abs;
// 此处异或运算必须加括号, 因为 verilog 中 + 的优先级更高
assign x_abs = ({32{x_signed}}^x) + x_signed;
assign y_abs = ({32{y_signed}}^y) + y_signed;
// 运算结果商的符号位, 做无符号除法时认为是 0
wire s_ref_signed = (x[31]^y[31]) & div_signed;
// 运算结果余数的符号位, 做无符号除法时认为是 0
wire r_ref_signed = x[31] & div_signed;

// 第二步, 求得商和余数的绝对值
reg [31:0] s_ref_abs;
reg [31:0] r_ref_abs;
always @(div_clk)
begin
    s_ref_abs <= x_abs/y_abs;
    r_ref_abs <= x_abs-s_ref_abs*y_abs;
end

// 第三步, 依据商和余数的符号位调整
wire [31:0] s_ref;
wire [31:0] r_ref;
// 此处异或运算必须加括号, 因为 Verilog 中加法的优先级更高
assign s_ref = ({32{s_ref_signed}}^s_ref_abs) + {30'd0,s_ref_signed};
assign r_ref = ({32{r_ref_signed}}^r_ref_abs) + r_ref_signed;
//-----{ 计算参考结果 }end

// 判断结果是否正确
wire s_ok;
wire r_ok;
assign s_ok = s_ref==s;
assign r_ok = r_ref==r;
reg [5:0] time_out;

// 输出结果, 将各 32 位 (不论是有符号还是无符号数) 扩展成 33 位有符号数,
// 以便以十进制形式打印
wire signed [32:0] x_d      = {div_signed&x[31],x};
wire signed [32:0] y_d      = {div_signed&y[31],y};
wire signed [32:0] s_d      = {div_signed&s[31],s};
wire signed [32:0] r_d      = {div_signed&r[31],r};
wire signed [32:0] s_ref_d = {div_signed&s_ref[31],s_ref};
wire signed [32:0] r_ref_d = {div_signed&r_ref[31],r_ref};
always @(posedge div_clk)
begin
    if (complete && div) // 除法完成
    begin
        if (s_ok && r_ok)
        begin
            $display("[time@%t]: x=%d, y=%d, signed=%d, "

```

```

        "s=%d, r=%d, s_OK=%b, r_OK=%b",
        $time,x_d,y_d,div_signed,s_d,r_d,s_ok,r_ok);
    end
    else
    begin
        $display("[time@t]Error: x=%d, y=%d, signed=%d, s=%d, "
        "r=%d, s_ref=%d, r_ref=%d, s_OK=%b, r_OK=%b",
        $time,x_d,y_d,div_signed,s_d,
        r_d,s_ref_d,r_ref_d,s_ok,r_ok);
        $finish;
    end
end
end
always @(posedge div_clk)
begin
    if (!resetn || !div_is_run || complete)
    begin
        time_out <= 6'd0;
    end
    else
    begin
        time_out <= time_out + 1'b1;
    end
end
end
always @(posedge div_clk)
begin
    if (time_out == 6'd34)
    begin
        $display("Error: div no end in 34 clk!");
        $finish;
    end
end
end
endmodule

```

至此，LoongArch32 精简版中用户态指令中所有的运算类指令都已经实现完毕。

6.3 转移指令的添加

在进行转移指令的添加工作之前，我们简要回顾一下简单流水线中与转移指令相关的设计要点：

- 1) 转移指令的功能有两个要素：一是决定是否跳转，二是决定跳转时的跳转目标。
- 2) LoongArch 指令集中的转移指令包括分支指令和跳转指令，前者是否跳转需

要进行判断，后者一定跳转。

- 3) LoongArch 指令集中转移指令的跳转目标的计算方式有两种：一是转移指令 PC 加上转移指令中的偏移立即数，二是通用寄存器中的值加上转移指令中的偏移立即数。
- 4) LoongArch 指令集中除了 Link 类转移指令外，均不会产生寄存器写动作。Link 类转移指令会将自身 PC 加 4（即返回地址）写入通用寄存器中。
- 5) 在单发射五级流水 CPU 中，将转移指令的处理放在译码级进行，当发生跳转时，在更新下一拍取指请求（nextPC）为跳转目标的同时，取消取指级可能取回的错误执行路径上的指令，以解决控制相关引起的流水线冲突。

上面的第 5 点告诉我们，对于新增的转移指令，其参与生成下一拍取指 PC 的操作还是要放到译码流水级进行。至于如何生成是否跳转的控制信号、如何生成跳转目标，以及是否需要向通用寄存器写入返回地址，现有的 CPU 中已经有处理框架。接下来，我们将梳理待实现的指令，分析它们如何在现有的处理框架下进行处理。

blt、bge、bltu 和 bgeu 指令的添加

通过分析 blt、bge、bltu 和 bgeu 指令的功能定义，可以得知它们的功能与 beq、bne 非常相似，区别仅在于是否跳转的判断条件。因此添加 blt、bge、bltu 和 bgeu 指令只需要对流水线中已有的转移指令是否跳转的判断逻辑进行扩展即可，即在 $GR[rj] = GR[rd]$ 、 $GR[rj] \neq GR[rd]$ 条件判断的基础上，增加 $GR[rj] <_{\text{signed}} GR[rd]$ 、 $GR[rj] \geq_{\text{signed}} GR[rd]$ 、 $GR[rj] <_{\text{unsigned}} GR[rd]$ 、 $GR[rj] \geq_{\text{unsigned}} GR[rd]$ 的条件判断支持，其余功能均可直接复用实现 beq 和 bne 的数据通路，相关控制信号的生成也可以参照 beq 和 bne 的控制信号进行。

至此，LoongArch32 精简版中所有的转移指令均已实现完毕。

6.4 访存指令的添加

6.4.1 ld.b、ld.h、ld.bu、ld.hu 指令的添加

分析 ld.b、ld.h、ld.bu、ld.hu 指令的功能定义并将其与 ld.w 的定义进行比较，可知：

- 1) 它们计算虚地址的操作数来源、地址计算方法、虚实地址映射的规则是完全一样的。
- 2) 它们得到的访存结果都是写回第 rd 项寄存器中。

3) 它们和 ld.w 指令的差异仅在于从内存取回的数据位宽不同。

再来考虑微结构方面的设计因素。因为数据 RAM 的位宽是 32 位, 所以这些指令访问数据 RAM 的地址都是用指令访存地址去掉最低两位得到的。

综上, 这四条指令在译码、执行、写回级的数据通路、控制逻辑可以完全复用 ld.w 指令的设计实现。下面讨论如何处理指令间存在的差异。

6.4.1.1 从数据 RAM 输出结果中选择所需内容

既然数据 RAM 的宽度是 4 字节, 那么 ld.b 和 ld.bu 访问的内容可以出现在这四个字节的任一个中, ld.h 和 ld.hu 访问的内容可以是其中的低 2 字节或高 2 字节。也就是说, 这些指令需要的数据并不总是出现在数据 RAM 输出数据的最低位置上。因此, 我们需要引入一个多路选择器。读者可能会问: 这应该是个右移字节的操作吗? 但请大家想一想, 移位器在实现时本质上是不是就是多路选择器?

这个多路选择器的选择信号是通过指令访存地址的最低两位以及访存操作的类型信息共同生成的。其设计原理是直截了当的, 请读者自行推导一下。

6.4.1.2 将选取内容扩展至 32 位

ld.b、ld.bu、ld.h 和 ld.hu 指令从数据 RAM 取回的数据宽度比通用寄存器的宽度要小, 所以这些数据需要扩展到 32 位才能成为最终写入寄存器的结果。进行符号扩展还是零扩展是根据指令来确定的, ld.b、ld.h 是符号扩展, ld.bu、ld.hu 是零扩展。那么为什么要分为符号扩展装载 (load) 和零扩展装载呢? 因为 C 语言里面有 signed char、unsigned char 以及 signed short、unsigned short, 但是加减等数值运算指令只有操作 32 位源操作数的版本, 所以数据从内存装载到寄存器中, 必须根据数据的 signed/unsigned 属性将其符号 / 零扩展至 32 位后才能进行后续的数值运算。尽管理论上可以定义字节、半字的符号扩展、零扩展指令来完成这里需要的扩展操作, 但是由于实际应用程序中这种扩展操作可能频繁出现, 而在内存取数之后顺带进行扩展处理资源开销也并不大, 所以实际上不少指令集都分别定义了符号扩展和零扩展的装载指令。

上述从数据 RAM 返回值中选取需要的内容以及将内容扩展至 32 位的数据通路都是要在 CPU 中新增加的。我们建议将这个数据通路放在访存流水级, 因为这些指令与 ld.w 一样都是最早在访存级向译码流水级进行前递, 这样译码流水级的阻塞控制信号几乎不用调整。从时间的角度来看, 新增的多选逻辑的电路延迟不是很大, 即使它位于关键路径上, 也不至于造成频率的大幅度下降。就本书希望读者达到的设计精细程度而言, 这种程度的频率损失就不需要进行设计上的调整了, 我们尽量保证设计的简洁。

最后提醒一下，在目前的设计中，访存地址的最低两位和访存操作的类型信息并没有从译码级或执行级传递到访存级，需要在数据通路中添加。

6.4.2 st.b、st.h 指令的添加

分析 st.b 和 st.h 指令的功能定义并将其与 st.w 的定义进行比较，可知：

- 1) 它们计算虚地址的操作数来源、地址计算方法、虚实地址映射规则是完全一样的。
- 2) 要存入数据 RAM 的数据都来自第 rd 项寄存器。
- 3) st.b 指令只向数据 RAM 写入 1 个字节，st.h 指令只向数据 RAM 写入 2 个字节。

根据上面的总结，实现 st.b 和 st.h 指令的关键在于如何在一块 4 字节宽的 RAM 上完成字节或半字的写入，这可以通过 RAM 的字节写使能来实现。所谓字节写使能，就是 RAM 每一项（或行）的每个字节都有自己单独的写使能。可知，在实现 st.b 指令写数据 RAM 的时候，如果地址最低两位等于 0b00，那么字节写使能就是 0b0001；如果地址最低两位等于 0b01，那么字节写使能就是 0b0010……在实现 st.h 指令的时候，如果地址的最低两位等于 0b00，那么字节写使能就是 0b0011。对于 st.w，字节写使能恒为 0b1111。

确定了字节写使能，接下来就是生成写数据了。以 st.b 指令为例，它写入内存的永远是第 rd 项寄存器的最低字节，但是它存入的未必是数据 RAM 中那一项的最低字节。有些读者马上想到可以把数据按字节为单位移动一下，于是下面的代码就出现了。

```
assign st_data = op_st_b ? (vaddr[1:0]==2'b00 ? {24'b0, rd_value[7:0]} :
                           vaddr[1:0]==2'b01 ? {16'b0, rd_value[7:0], 8'b0} :
                           vaddr[1:0]==2'b10 ? {8'b0, rd_value[7:0], 16'b0} :
                           {rd_value[7:0], 24'b0}
                           ) :
op_st_h ? (vaddr[1:0]==2'b00 ? {16'b0, rd_value[15:0]} :
           {rd_value[15:0], 16'b0}
           ) :
rd_value;
```

上面的代码对不对？对。上面的代码效果好不好？一般。

其实，下面这样的代码也是对的。读者可以自行推演一下。提示一下，要考虑字节写使能。

```
assign st_data = op_st_b ? {4{rd_value[ 7:0]}} :
op_st_h ? {2{rd_value[15:0]}} :
rd_value[31:0];
```

后面这段代码是不是简洁多了？而且这段代码逻辑更少，时序也更好。所以，写代码和写文章是一样的，好的作品都需要花心思推敲、琢磨。

至此，本章所要添加的普通用户态指令已经都实现完毕。

6.5 任务与实践

完成本章的学习后，希望读者能够完成以下 2 个实践任务：

- 1) 算术逻辑运算指令和乘除法运算指令添加，参见 6.5.1 节。
- 2) 转移指令和访存指令添加，参见 6.5.2 节。

6.5.1 实践任务 10：算术逻辑运算指令和乘除法运算指令添加

本实践任务要求在实践任务 9 实现的 CPU 基础上完成以下工作：

- 1) 添加算术逻辑运算类指令 `slti`、`sltui`、`andi`、`ori`、`xori`、`sll`、`srl`、`sra`、`pcaddu12i`。
- 2) 添加乘除运算类指令 `mul.w`、`mulh.w`、`mulh.wu`、`div.w`、`mod.w`、`div.wu`、`mod.wu`。
- 3) 运行 `exp10` 对应的 `func`，要求成功通过仿真和上板验证。

请参照 2.3.1 节中介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环境仍位于 `mycpu_env/` 目录下，且仍使用 `soc_bram/` 子目录。

实验环境准备就绪后，请参考下列步骤完成本实践任务：

- 1) 将所实现 CPU 的代码更新至 `mycpu_env/myCPU/` 目录中。
- 2) 修改 `func` 配置文件——`mycpu_env/func/include/test_config.h`，选择 `exp10` 的配置，编译。（如果是通过压缩包 `exp10.zip` 获取实验开发环境的，请跳过该步骤。）
- 3) 打开 `gettrace` 工程——`mycpu_env/gettrace/gettrace.xpr`。（该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的，如果使用更高版本的 Vivado 打开，请参考附录 D.4 节进行 IP 核升级。）运行 `gettrace` 工程的仿真（进入仿真界面后，直接点击 `run all` 等待仿真运行完成），生成新的参考 `trace` 文件 `golden_trace.txt`（`mycpu_env/gettrace/golden_trace.txt`）。要等仿真运行完成，`golden_trace.txt` 才有完整的内容。（如果是通过压缩包 `exp10.zip` 获取实验开发环境的，请跳过该步骤。）
- 4) 进入 `mycpu_env/soc_verify/soc_bram/run_vivado/` 目录下启动验证 `myCPU` 的工程。如果该目录下尚未创建工程，请参考附录 D.2 节介绍的步骤，利用该目录

下的 `create_project.tcl` 文件创建工程。如需要, 请参考附录 D.4 节进行 IP 核升级。如果该目录下已有前一实践任务创建过的工程, 可以在打开工程后, 参照附录 D.3 节介绍的步骤, 更新项目中 CPU 实现文件的列表。

- 5) 参考 4.4.5.2 节, 对工程中的 `inst_ram` 重新定制。(如果是通过压缩包 `exp10.zip` 获取实验开发环境的, 请跳过该步骤。)
- 6) 在验证 `myCPU` 的工程中运行仿真(进入仿真界面后, 直接点击 `run all`), 进行功能验证与调试, 直至仿真测试通过。
- 7) 在验证 `myCPU` 的工程中综合实现后生成二进制码流文件, 进行上板验证。(如无硬件实验平台, 请跳过该步骤。)

6.5.2 实践任务 11: 转移指令和访存指令添加

本实践任务要求在实践任务 10 实现的 CPU 基础上完成以下工作:

- 1) 添加转移指令 `blt`、`bge`、`bltu`、`bgeu`。
- 2) 添加访存指令 `ld.b`、`ld.h`、`ld.bu`、`ld.hu`、`st.b`、`st.h`。
- 3) 运行 `exp11` 对应的 `func`, 要求成功通过仿真和上板验证。

请参照 2.3.1 节中介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环境仍位于 `mycpu_env/` 目录下, 且仍使用 `soc_bram/` 子目录。

实验环境准备就绪后, 请参考下列步骤完成本实践任务:

- 1) 将所实现 CPU 的代码更新至 `mycpu_env/myCPU/` 目录中。
- 2) 修改 `func` 配置文件——`mycpu_env/func/include/test_config.h`, 选择 `exp11` 的配置, 编译。(如果是通过压缩包 `exp11.zip` 获取实验开发环境的, 请跳过该步骤。)
- 3) 打开 `gettrace` 工程——`mycpu_env/gettrace/gettrace.xpr`。(该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的, 如果使用更高版本的 Vivado 打开, 请参考附录 D.4 节进行 IP 核升级。)运行 `gettrace` 工程的仿真(进入仿真界面后, 直接点击 `run all` 等待仿真运行完成), 生成新的参考 `trace` 文件 `golden_trace.txt` (`mycpu_env/gettrace/golden_trace.txt`)。要等仿真运行完成, `golden_trace.txt` 才有完整的内容。(如果是通过压缩包 `exp11.zip` 获取实验开发环境的, 请跳过该步骤。)
- 4) 进入 `mycpu_env/soc_verify/soc_bram/run_vivado/` 目录下启动验证 `myCPU` 的工程。如果该目录下尚未创建工程, 请参照附录 D.2 节介绍的步骤, 利用该目录下的 `create_project.tcl` 文件创建工程。如需要, 请参考附录 D.4 节进行 IP 核升

级。如果该目录下已有前一实践任务创建过的工程，可以在打开工程后，参照附录 D.3 节介绍的步骤，更新项目中 CPU 实现文件的列表。

- 5) 参考 4.4.5.2 节，对工程中的 `inst_ram` 重新定制。（如果是通过压缩包 `exp11.zip` 获取实验开发环境的，请跳过该步骤。）
- 6) 在验证 `myCPU` 的工程中运行仿真（进入仿真界面后，直接点击 `run all`），进行功能验证与调试，直至仿真测试通过。
- 7) 在验证 `myCPU` 的工程中综合实现后生成二进制码流文件，进行上板验证。（如无硬件实验平台，请跳过该步骤。）